```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go


%matplotlib inline

import warnings
warnings.filterwarnings('ignore')
```

```
superstore_sales = pd.read_excel('US Superstore data.xls')
superstore_sales.sample(8)
```

EXPLORATORY DATA

```
df = superstore_sales.copy()
```

```
df.shape
```

```
df.columns
```

```
df.info()
```

```
df[['Sales','Discount','Profit','Quantity']].describe()
```

```
df=df.drop('Row ID',axis=1)
```

```
df['Country'].value_counts()
```

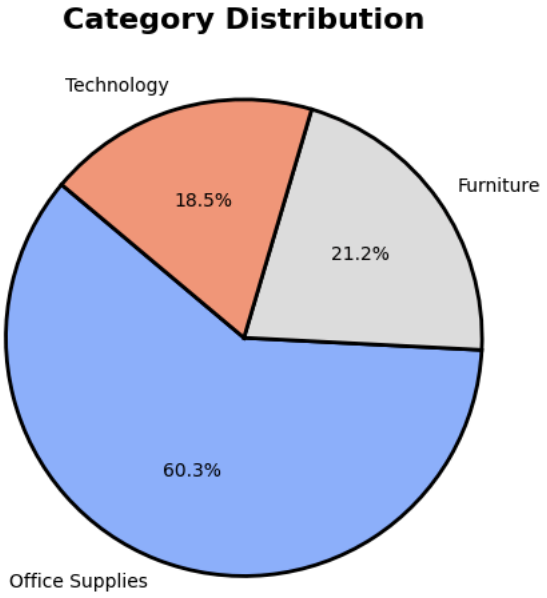
```
df=df.drop('Country',axis=1)
```

```
df.drop(['Order ID',         'Ship Date',          'Ship Mode',          'Customer ID',          'Customer Name',
```

```
df.shape
```

```
category_counts = df['Category'].value_counts()
category_counts
```

```
fig, axs = plt.subplots(figsize=(5, 5))

axs.pie(
    category_counts,
    labels=category_counts.index,
    autopct='%1.1f%%',
    startangle=140,
    colors=sns.color_palette('coolwarm', len(category_counts)),
    wedgeprops={'edgecolor': 'black', 'linewidth': 2}
)

axs.set_title('Category Distribution', fontsize=16, fontweight='bold')

plt.tight_layout()
plt.show()
```

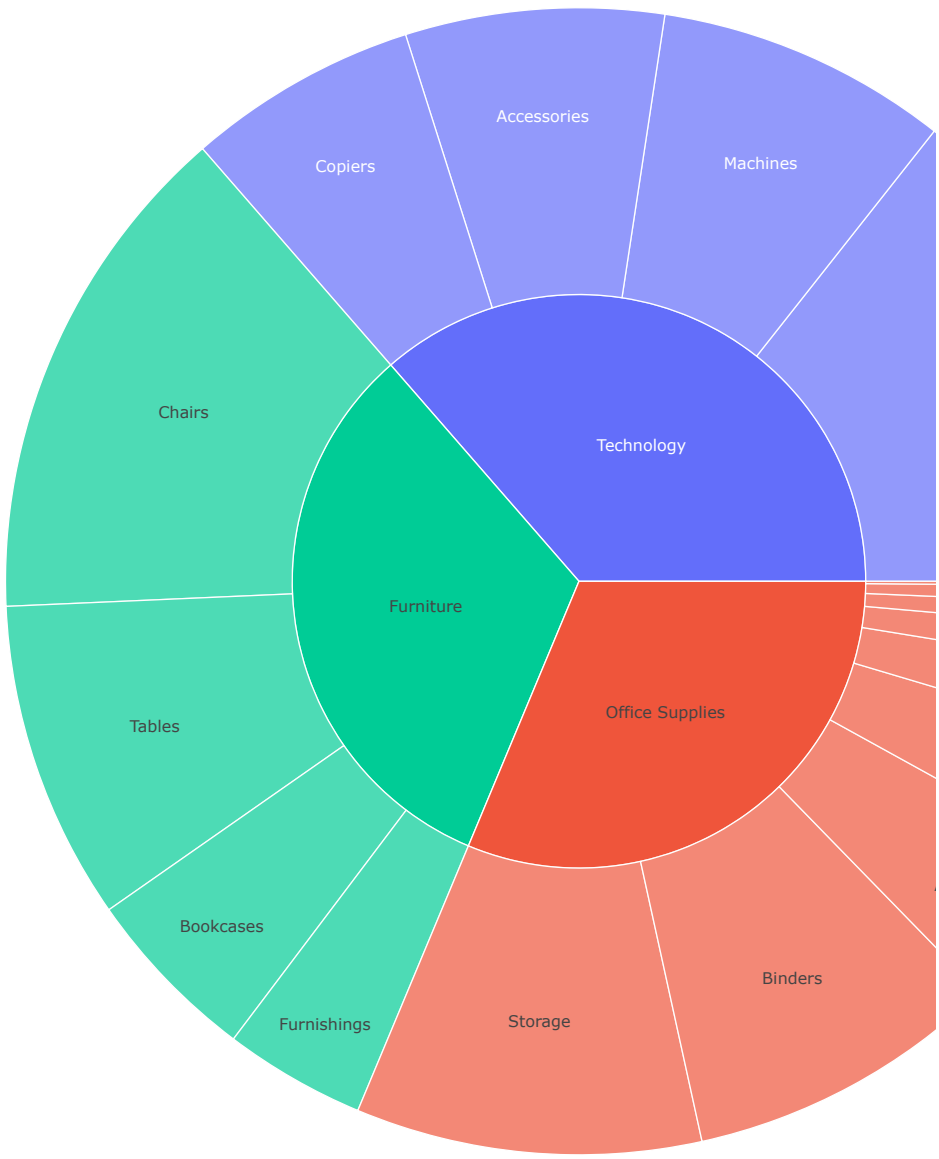


Category Distribution (Pie Chart)

The pie chart shows how your data is distributed across different product categories. One or two categories likely dominate the dataset, contributing the largest share. Smaller slices indicate less frequent categories or lower representation. This helps quickly identify which category is most common or important in your dataset.

Insight: If one category is significantly larger, your business/data is heavily dependent on that segment. Balanced slices indicate diversification across categories.

```
import plotly.express as px

fig = px.sunburst(
    df,
    path=['Category', 'Sub-Category'],
    values='Sales',
    color='Category',
    hover_data=['Sales', 'Quantity', 'Profit']
)

fig.update_layout(
    height=1000,
    title_text='Product Categories & Sub-Categories'
)

fig.show()
```

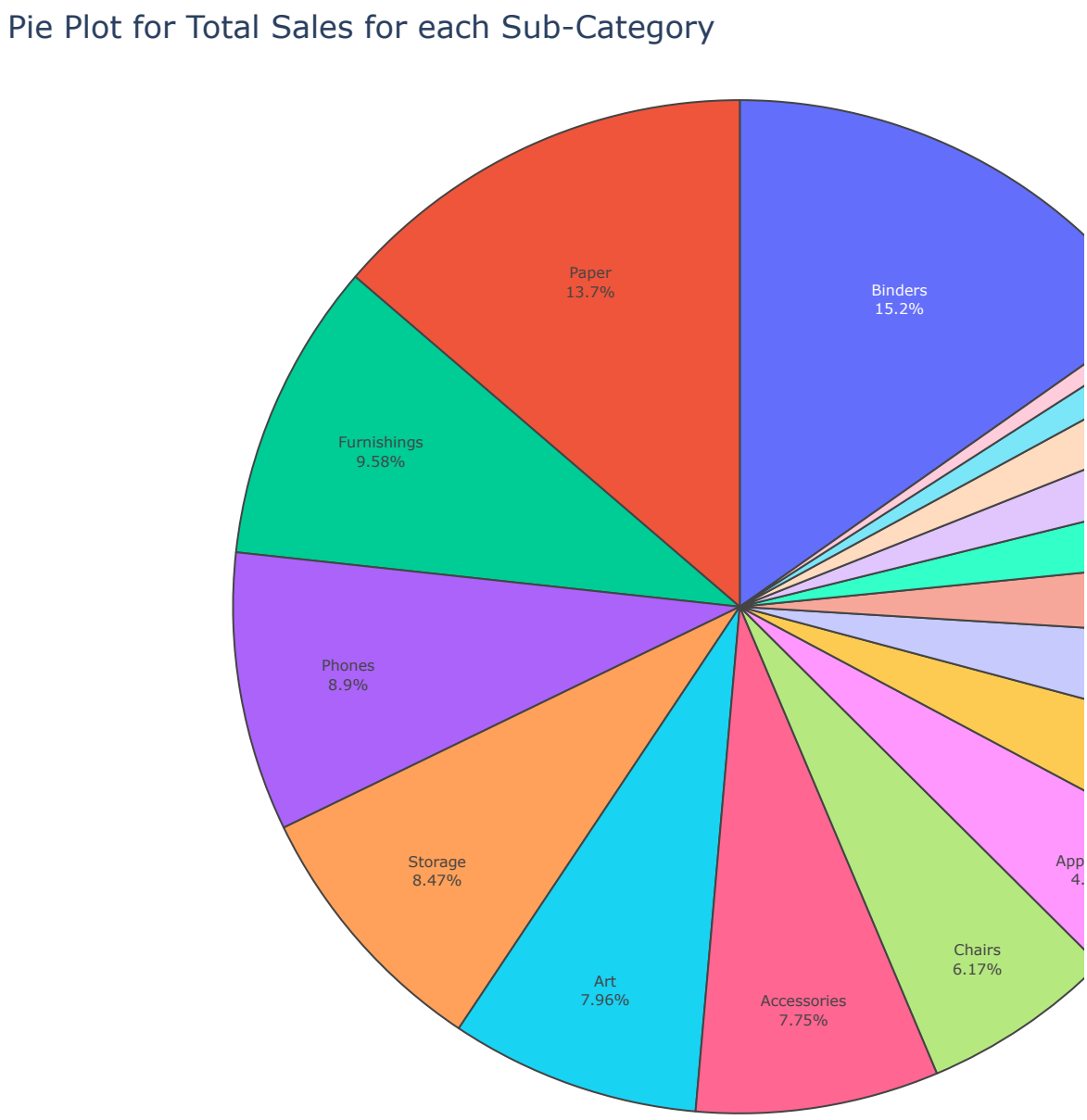


Category → Sub-Category (Sunburst Chart)

The sunburst chart gives a hierarchical breakdown: Inner ring → Category Outer ring → Sub-Category The size of each segment represents Sales contribution. Hovering shows detailed metrics like Sales, Quantity, and Profit.

Insight: One can identify Which sub-categories drive the most sales Which ones contribute less (or may need improvement) It reveals hidden patterns not visible in the pie chart (like which sub-category dominates inside a category)

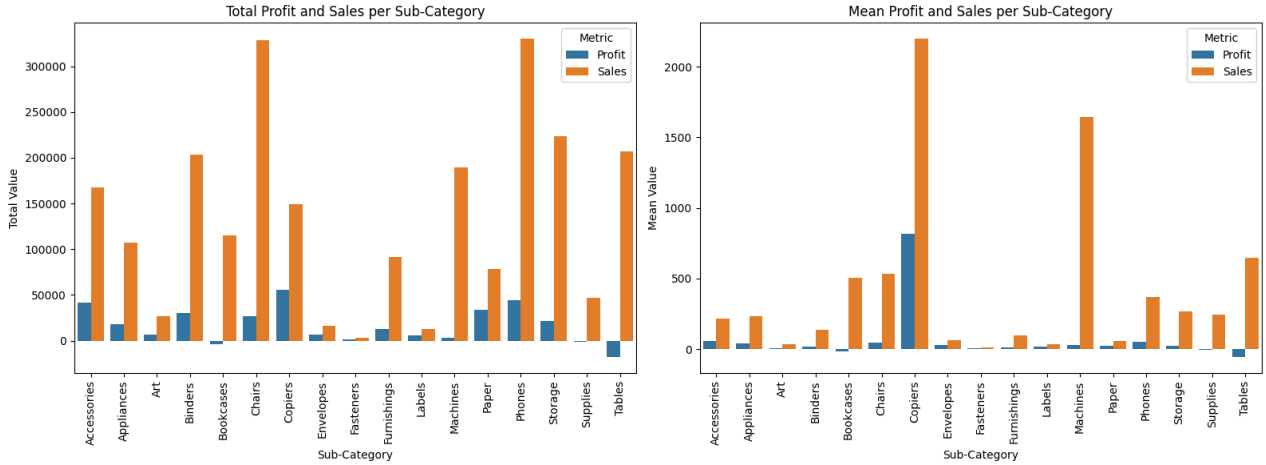
```
import plotly.graph_objects as go
from plotly.offline import iplot

data = go.Pie(
    labels=df['Sub-Category'].value_counts().index,
    values=df['Sub-Category'].value_counts(),
    textinfo='label+percent',
    hoverinfo='label+percent',
    marker=dict(line=dict(width=1.5))
)

layout = go.Layout(
    title='Pie Plot for Total Sales for each Sub-Category',
    titlefont=dict(size=25),
    height=1000
)

iplot(dict(data=[data], layout=layout))
```



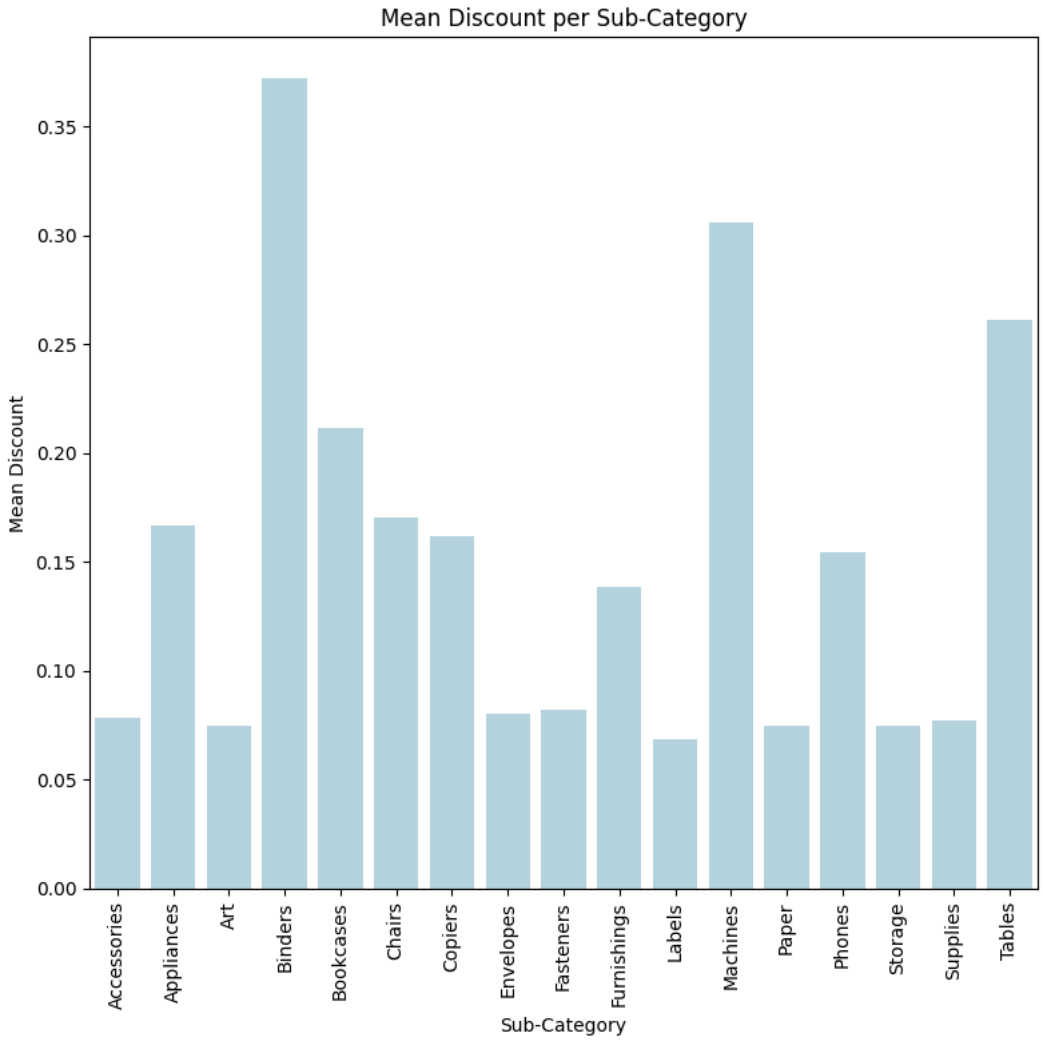
Review of sales and profits

```
# Grouping
grouped_sum = df.groupby('Sub-Category')[['Profit', 'Sales']].sum().reset_index()
grouped_mean = df.groupby('Sub-Category')[['Profit', 'Sales']].mean().reset_index()

# Create subplots
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Total values plot
sns.barplot(
    data=grouped_sum.melt(id_vars='Sub-Category', var_name='Metric', value_name='Value'),
    x='Sub-Category',
    y='Value',
    hue='Metric',
    ax=axes[0]
)
axes[0].set_title('Total Profit and Sales per Sub-Category')
axes[0].set_ylabel('Total Value')
axes[0].tick_params(axis='x', rotation=90)

# Mean values plot
sns.barplot(
    data=grouped_mean.melt(id_vars='Sub-Category', var_name='Metric', value_name='Value'),
    x='Sub-Category',
    y='Value',
    hue='Metric',
```

```
    ax=axes[1]
)
axes[1].set_title('Mean Profit and Sales per Sub-Category')
axes[1].set_ylabel('Mean Value')
axes[1].tick_params(axis='x', rotation=90)

plt.tight_layout()
plt.show()
```



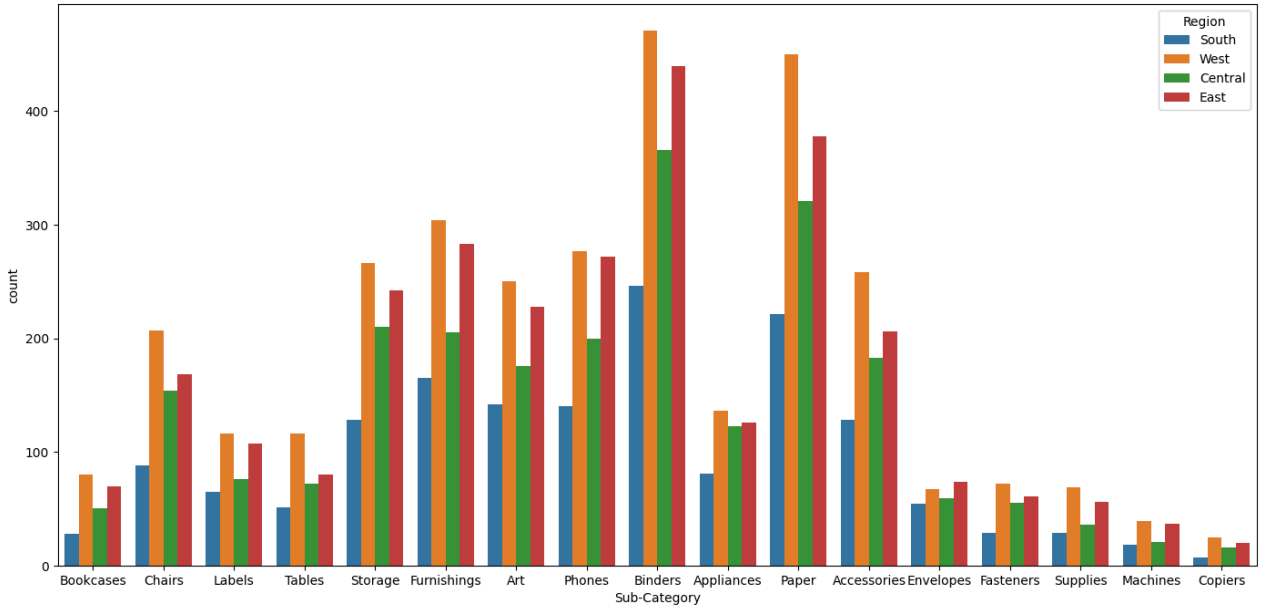

```
grouped_discount_mean = df.groupby('Sub-Category')['Discount'].mean().reset_index()

fig, ax = plt.subplots(figsize=(8, 8))

sns.barplot(
    data=grouped_discount_mean,
    x='Sub-Category',
    y='Discount',
    ax=ax,
    color='lightblue'
)

ax.set_title('Mean Discount per Sub-Category')
ax.set_ylabel('Mean Discount')
ax.tick_params(axis='x', rotation=90)

plt.tight_layout()
plt.show()
```



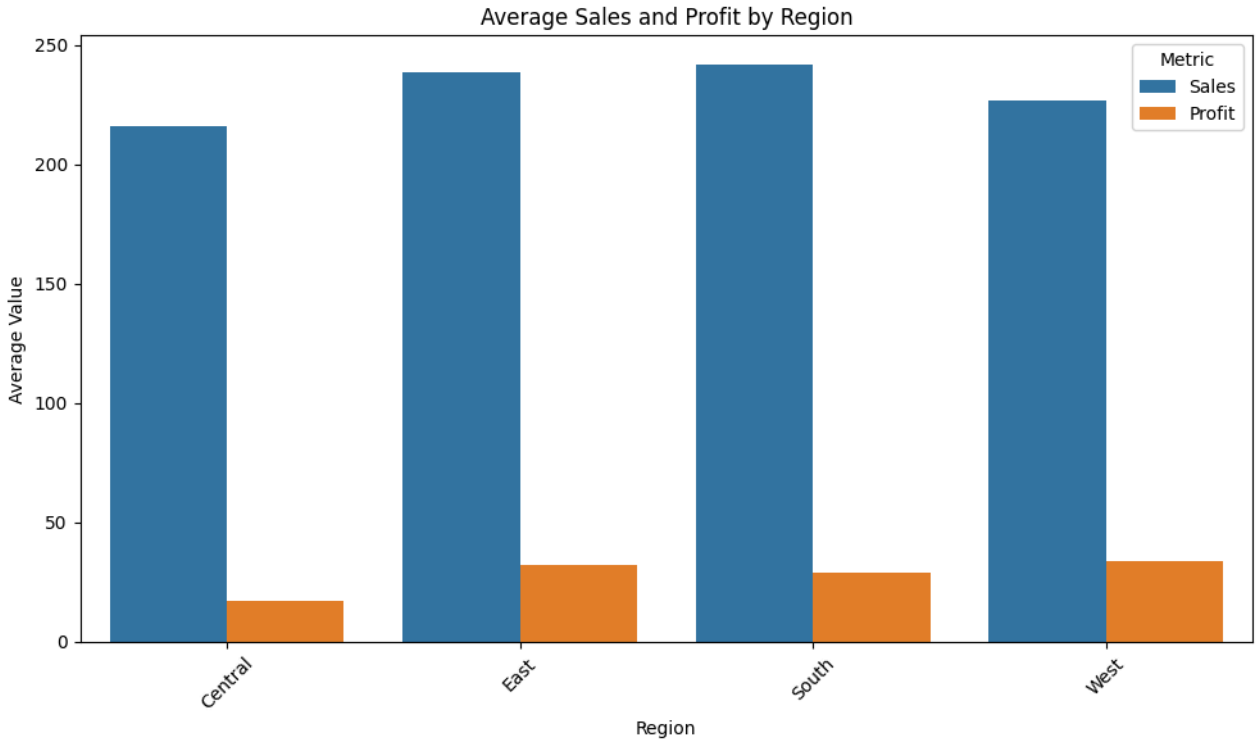
;

Review of the sales of different parts of the US

```
plt.figure(figsize=(17,8))
sns.countplot(x="Sub-Category", hue="Region", data=df)
plt.show()
```

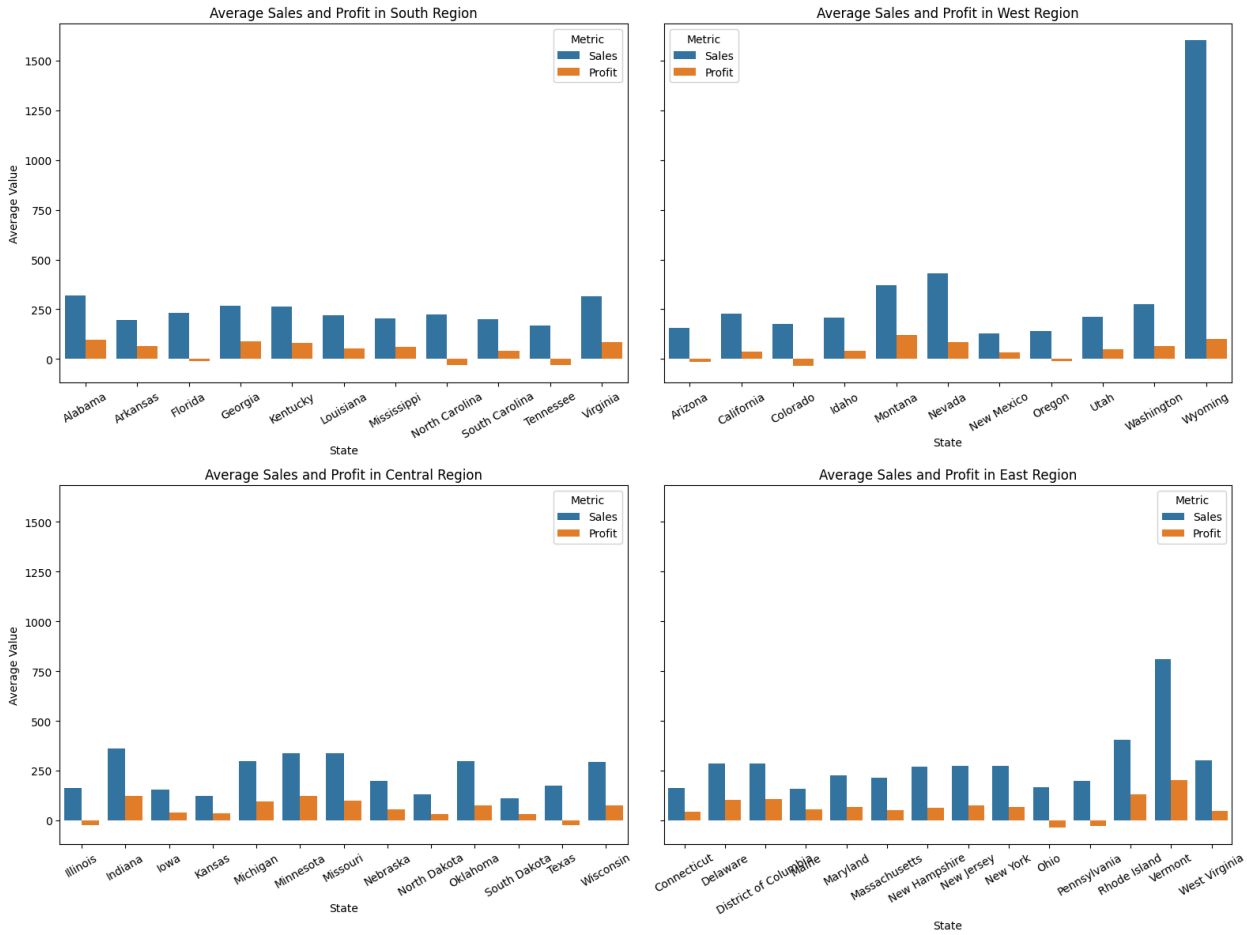



```
grouped_data = df.groupby('Region')[['Sales', 'Profit']].mean().reset_index()

melted_data = grouped_data.melt(
    id_vars='Region',
    var_name='Metric',
    value_name='Value'
)

plt.figure(figsize=(10, 6))

sns.barplot(
    data=melted_data,
    x='Region',
    y='Value',
    hue='Metric'
)

plt.title('Average Sales and Profit by Region')
plt.ylabel('Average Value')
plt.xlabel('Region')

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

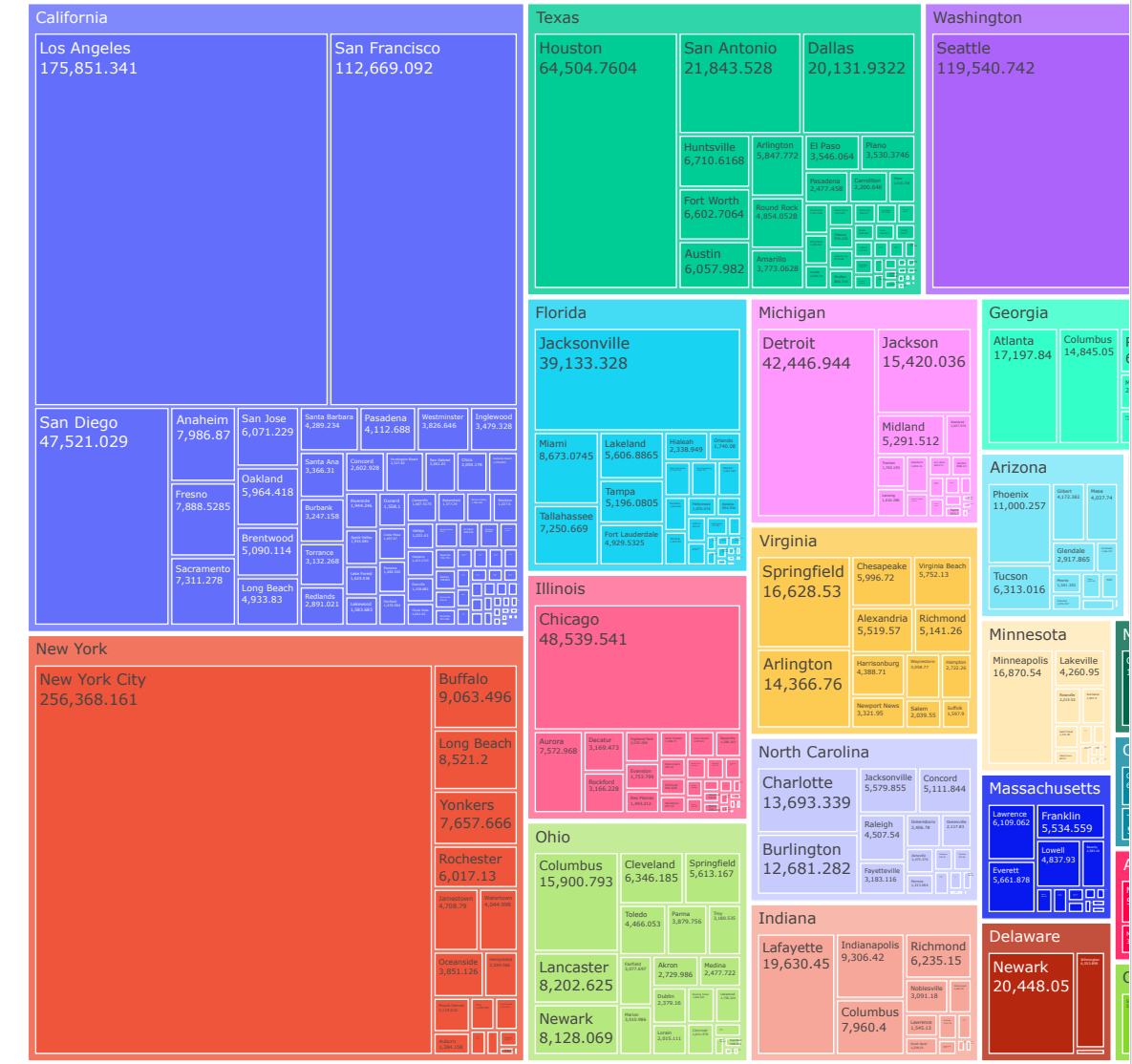


|||

```
regions = df['Region'].unique()

fig, axes = plt.subplots(2, 2, figsize=(16, 12), sharey=True)
axes = axes.flatten()

for i in range(4):
    df_region = df[df['Region'] == regions[i]].copy()

    grouped_data = df_region.groupby('State')[['Sales', 'Profit']].mean().reset_index()

    melted_data = grouped_data.melt(
        id_vars='State',
        var_name='Metric',
        value_name='Value'
    )

    sns.barplot(
        data=melted_data,
        x='State',
        y='Value',
        hue='Metric',
        ax=axes[i]
    )

    axes[i].set_title(f'Average Sales and Profit in {regions[i]} Region')
    axes[i].set_ylabel('Average Value')
    axes[i].tick_params(axis='x', rotation=30)
    axes[i].legend(title='Metric')

plt.tight_layout()
plt.show()
```

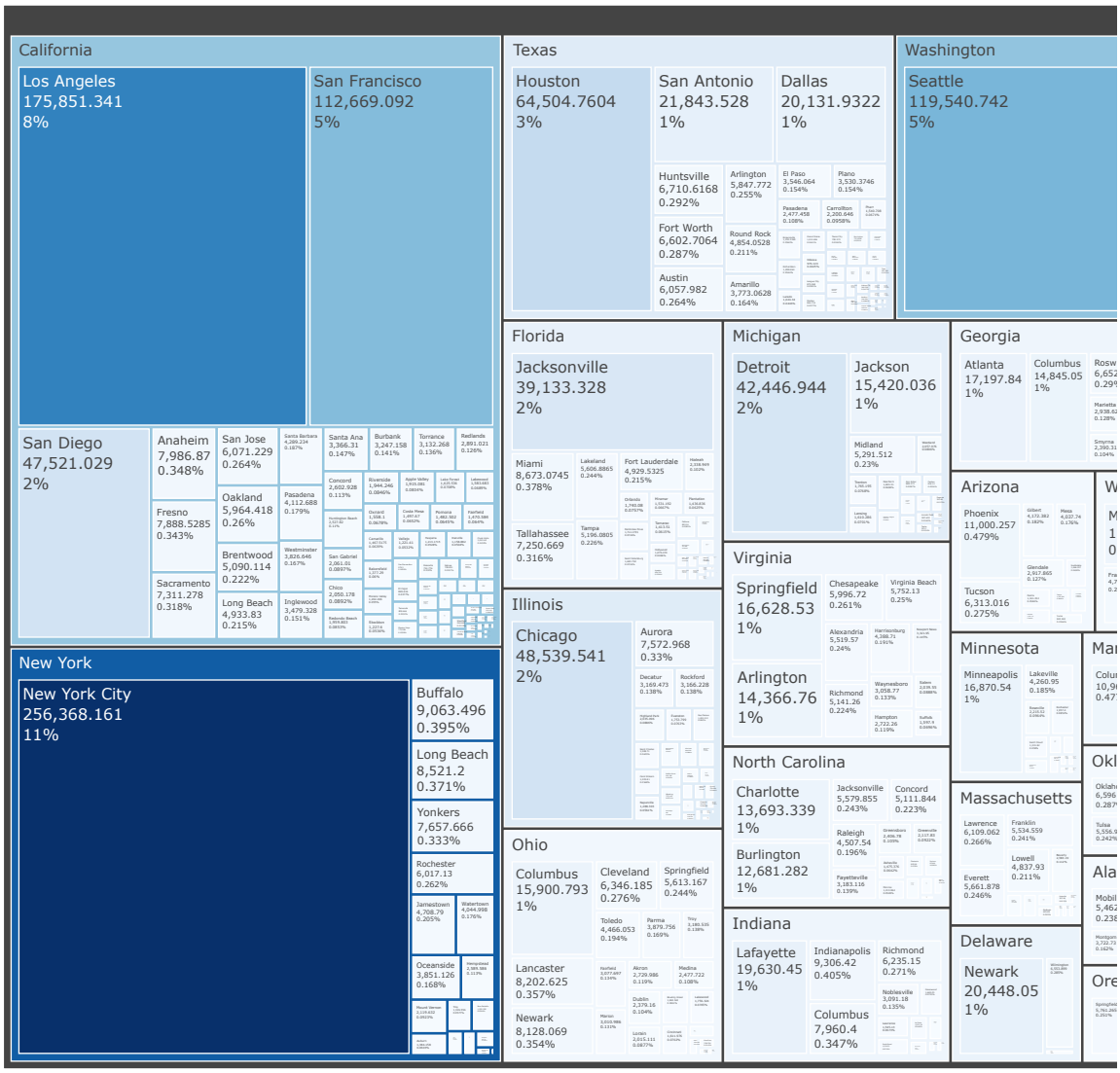



```
import plotly.express as px

temp = df[['State', 'City', 'Sales']] \
    .groupby(['State', 'City'])['Sales'] \
    .sum() \
    .reset_index()

fig = px.treemap(
    temp,
    path=['State', 'City'],
    values='Sales'
)

fig.update_layout(
    height=1000,
    title='City-wise Sales'

)

fig.data[0].textinfo = 'label+text+value'

fig.show()
```




```
temp = df[['State', 'City', 'Sales']] \
    .groupby(['State', 'City'])['Sales'] \
    .sum() \
    .reset_index()

fig = px.treemap(
    temp,
    path=['State', 'City'],
    values='Sales',
    color='Sales',
    color_continuous_scale='Blues',
    title='City-wise Sales Distribution'
)

fig.update_layout(
    height=1000
)


fig.data[0].textinfo = 'label+value+percent entry'

fig.update_traces(
    hovertemplate='<b>%{label}</b><br>Sales: %{value}<br>Share: %{percentEntry:.2%}<extra></extra>'
)

fig.show()
```

City-wise Sales Distribution

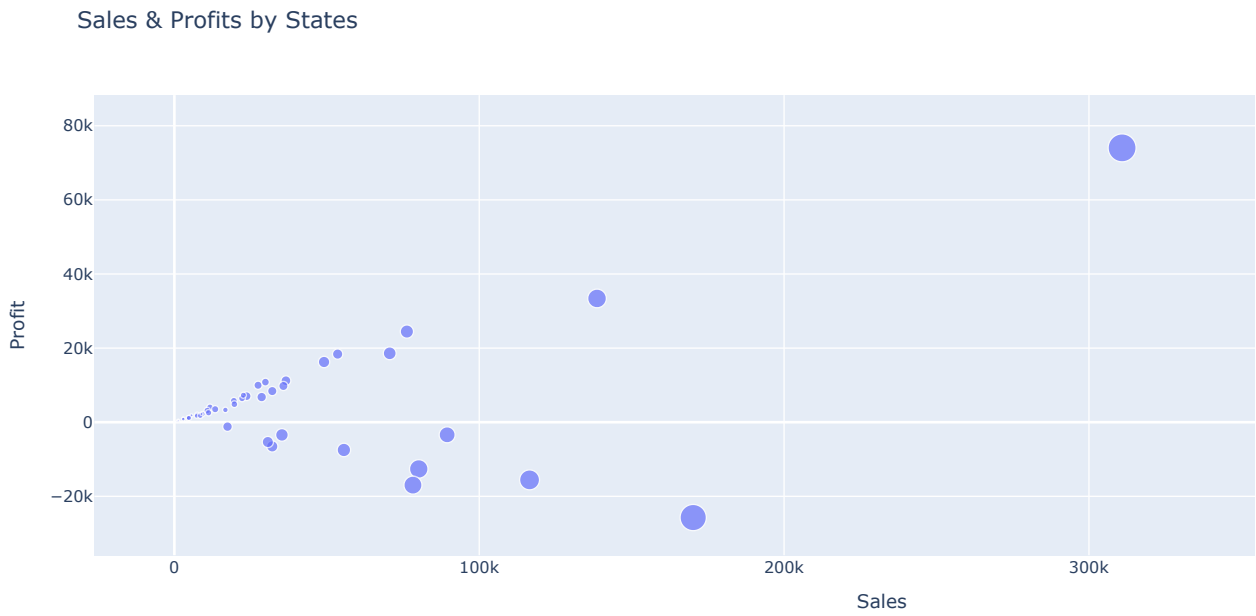


uuuu

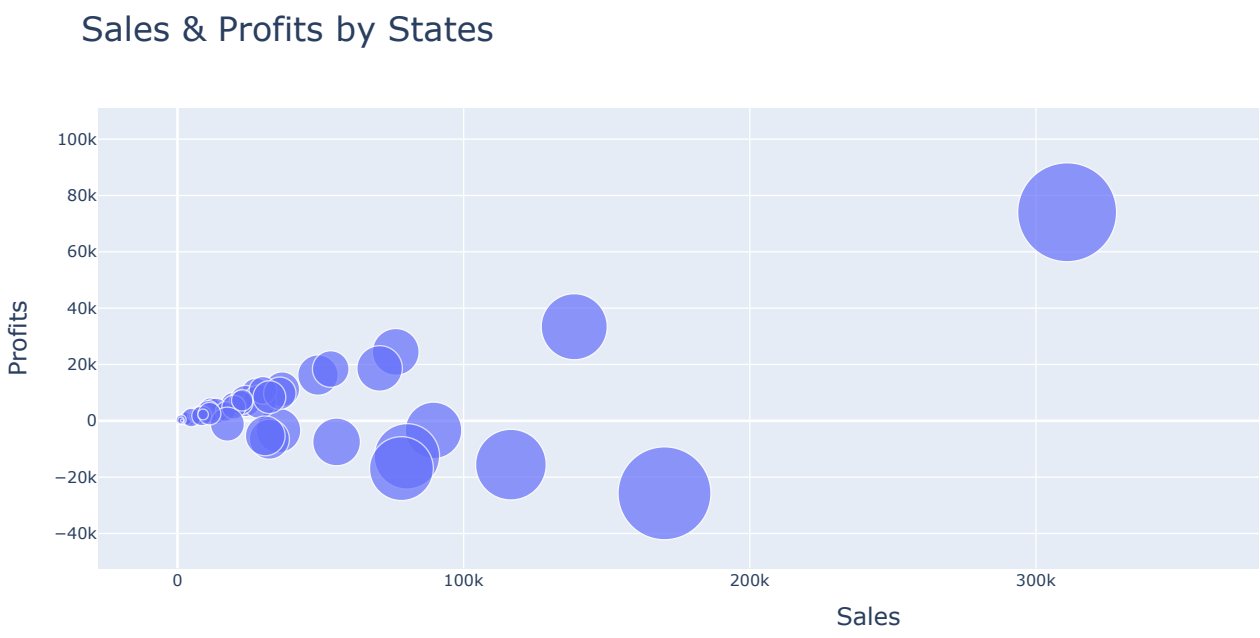
```
df_notime = df[['Segment', 'City', 'State', 'Region', 'Category', 'Sub-Category', 'Sales', 'Quantity', 'Discount', 'Profit
```

```
df
```

```
state_data = df_notime.groupby('State').sum().reset_index()

fig = px.scatter(
    state_data,
    x='Sales',
    y='Profit',
    size='Quantity',
    hover_name='State',
    title='Sales & Profits by States'
)

fig.show()
```

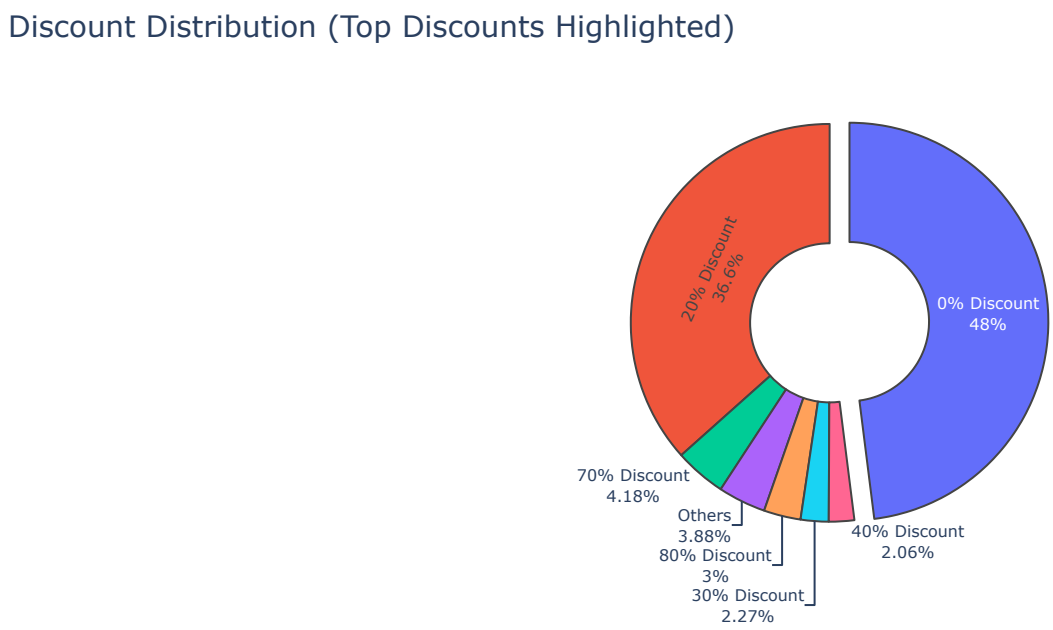



```
#Advanced Analysis

state_data = df_notime.groupby('State').sum().reset_index()

fig = go.Figure(
    data=go.Scatter(
        x=state_data['Sales'],
        y=state_data['Profit'],
        mode='markers',
        text=state_data['State'],
        marker=dict(
            size=state_data['Quantity'],
            sizemode='area',
            sizeref=2.*max(state_data['Quantity'])/(100.**2),
            showscale=True
        )
    )
)

fig.update_layout(
    title='Sales & Profits by States',
    title_font=dict(size=25),
    xaxis=dict(title='Sales', title_font=dict(size=18)),
    yaxis=dict(title='Profits', title_font=dict(size=18))
)

fig.show()
```



Discount Analysis

```
discount_counts = df_notime['Discount'].value_counts().sort_values(ascending=False)


top_n = 6
top_discounts = discount_counts.head(top_n)
others = discount_counts.iloc[top_n:].sum()


values = list(top_discounts.values)
labels = [f"{int(d*100)}% Discount" for d in top_discounts.index]

if others > 0:
    values.append(others)
    labels.append('Others')


fig = go.Figure(
    data=[go.Pie(
        labels=labels,
        values=values,
        hole=0.4,
        textinfo='label+percent',
        hoverinfo='label+percent+value',
        marker=dict(line=dict(width=1.5)),
        pull=[0.1 if i == 0 else 0 for i in range(len(labels))]
    )]
)

fig.update_layout(
    title='Discount Distribution (Top Discounts Highlighted)',
    title_font=dict(size=22)
)

fig.show()
```




```
discount_counts = df_notime['Discount'].value_counts().sort_index()
```

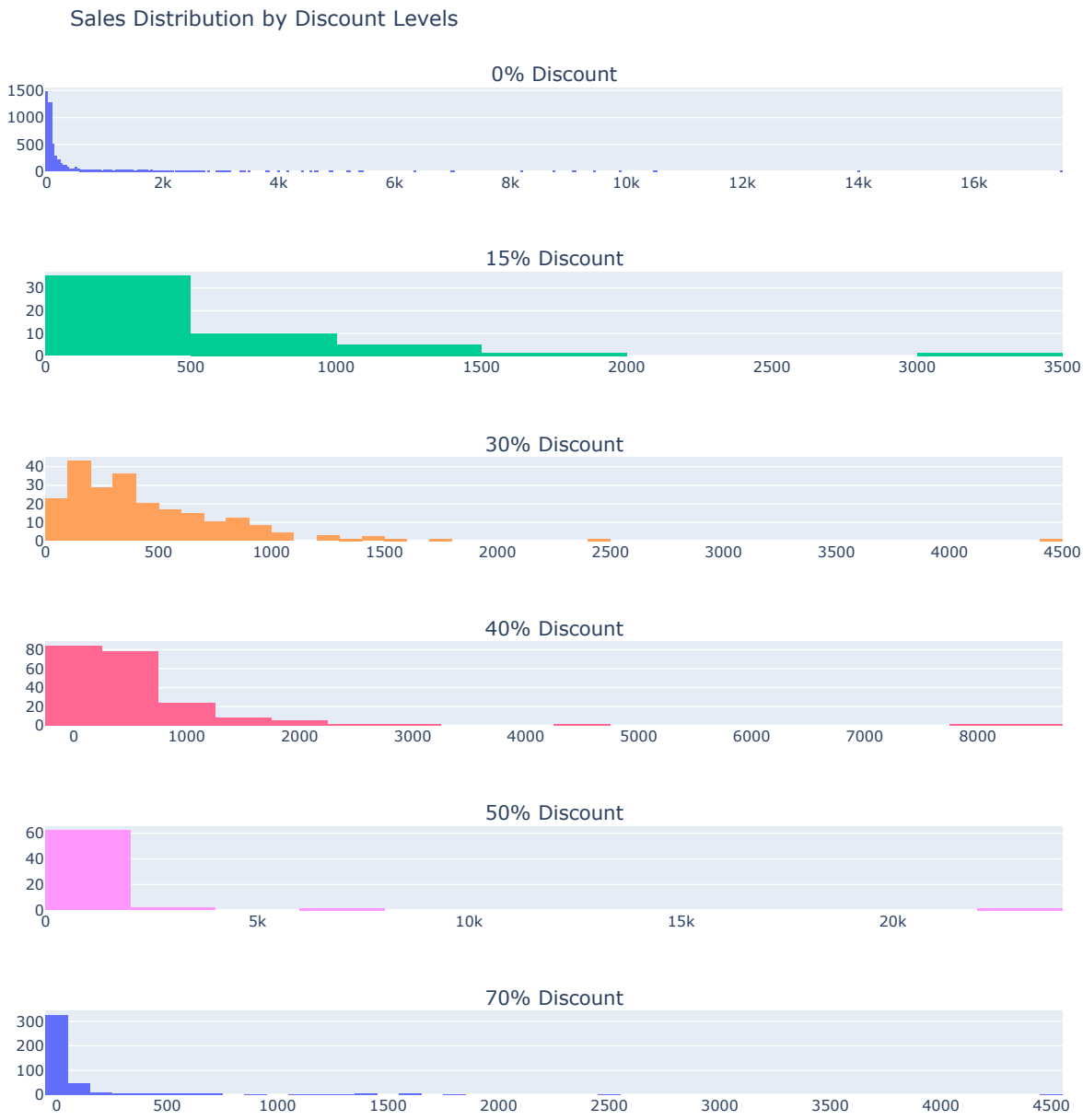
A few discount levels dominate most transactions

Remaining discount levels contribute relatively less and are grouped as “Others”

Helps identify most frequently used pricing strategies

```
import plotly.graph_objects as go

data = [
    go.Histogram(
        x=df_notime.sort_values('Discount')['Discount']
    )
]

layout = go.Layout(
    title='Sales Histogram Plot For Each Discount',
    xaxis=dict(
        title='Discount',
        title_font=dict(size=20)
    )
)

fig = dict(data=data, layout=layout)

iplot(fig)
```

```
from plotly.subplots import make_subplots

fig = make_subplots(
    rows=6,
    cols=2,
    subplot_titles=[
        '0% Discount','10% Discount','15% Discount',
        '20% Discount','30% Discount','32% Discount',
        '40% Discount','45% Discount','50% Discount',
        '60% Discount','70% Discount','80% Discount'
    ],
    horizontal_spacing=0.1,
    vertical_spacing=0.1
)

fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0]['Sales'], name='0% Distribution'), row=1, col=1)
fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.1]['Sales'], name='10% Distribution'), row=1, col=2)

fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.15]['Sales'], name='15% Distribution'), row=2, col=1)
fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.2]['Sales'], name='20% Distribution'), row=2, col=2)

fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.3]['Sales'], name='30% Distribution'), row=3, col=1)
fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.32]['Sales'], name='32% Distribution'), row=3, col=2)

fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.4]['Sales'], name='40% Distribution'), row=4, col=1)
fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.45]['Sales'], name='45% Distribution'), row=4, col=2)

fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.5]['Sales'], name='50% Distribution'), row=5, col=1)
fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.6]['Sales'], name='60% Distribution'), row=5, col=2)

fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.7]['Sales'], name='70% Distribution'), row=6, col=1)
fig.add_trace(go.Histogram(x=df_notime[df_notime['Discount']==0.8]['Sales'], name='80% Distribution'), row=6, col=2)

fig.update_layout(
    width=2000,
    height=1000,
    title_text='Sales Distribution by Discount Levels',
    showlegend=False
)

fig.show()
```



By applying the right policy on discounts,the loss of the store can be avoided in some sections

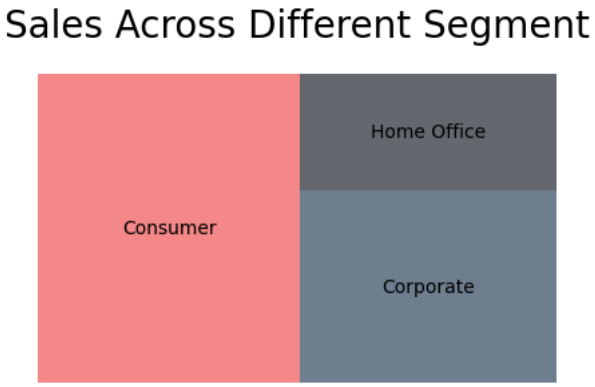
SEGMENTATION

```
!pip install squarify
```

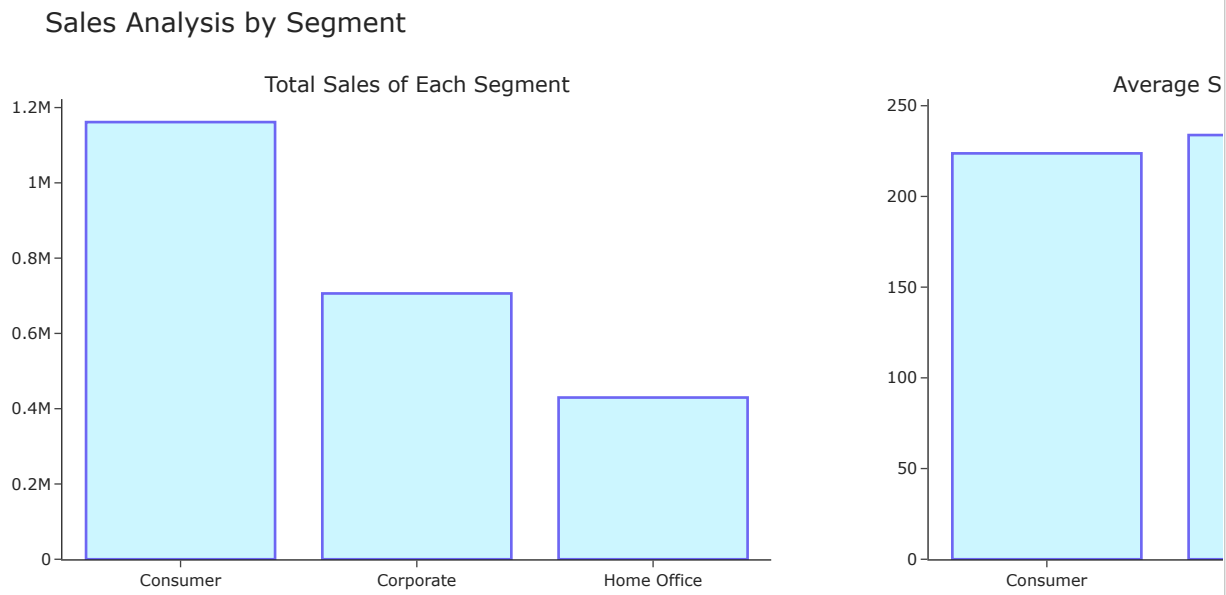
```
Collecting squarify
  Downloading squarify-0.4.4-py3-none-any.whl.metadata (600 bytes)
Downloading squarify-0.4.4-py3-none-any.whl (4.1 kB)
Installing collected packages: squarify
Successfully installed squarify-0.4.4
```

```
import squarify

df_tree = df[['Segment', 'Sales']]
df_tree = df_tree.groupby(['Segment']).sum().reset_index()

plt.figure(figsize=(5, 3))

squarify.plot(
    sizes=df_tree['Sales'],
    label=df_tree['Segment'],
    color=['#F05454', '#30475E', '#222831'],
```

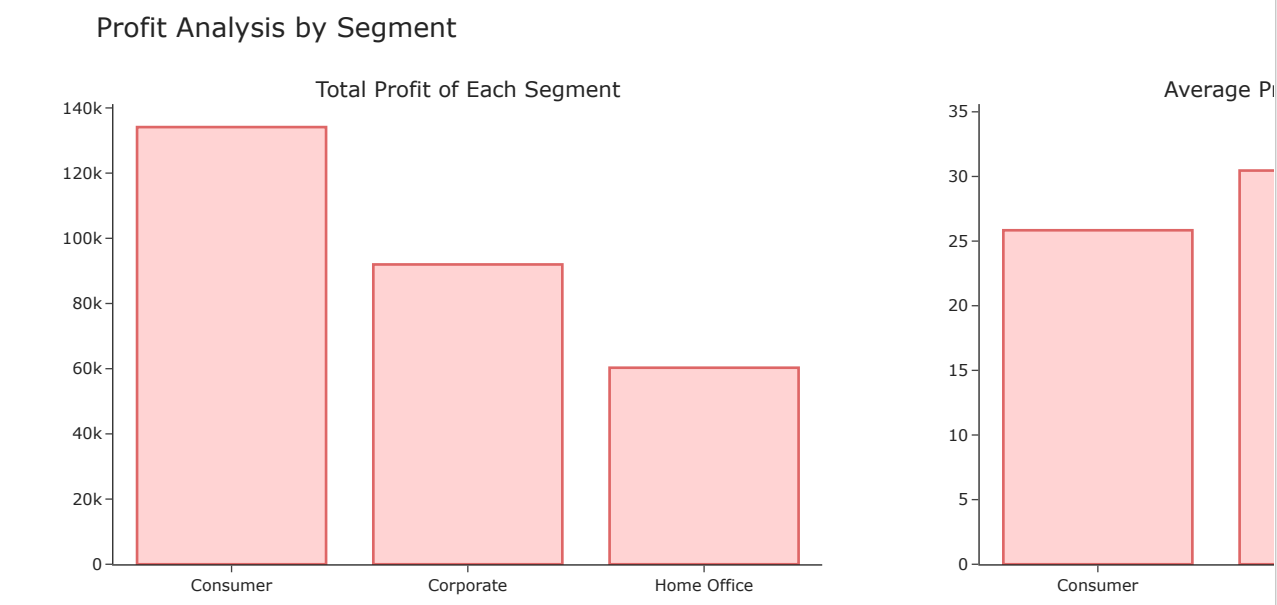
```
    alpha=0.7
)

plt.title("Sales Across Different Segment", size=20, pad=20)
plt.axis('off')

plt.show()
```



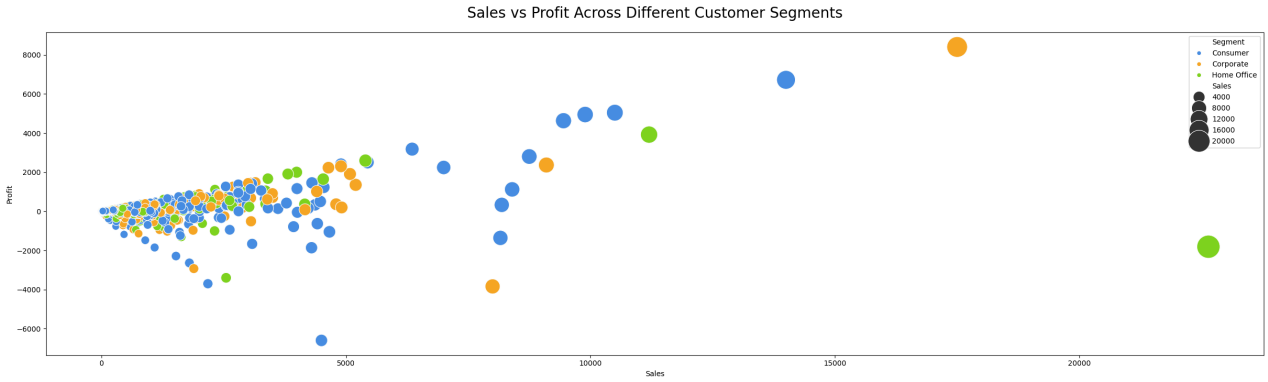

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go


segments = df['Segment'].unique()


total_sales = [df[df['Segment'] == i]['Sales'].sum() for i in segments]
average_sales = [df[df['Segment'] == i]['Sales'].mean() for i in segments]


fig = make_subplots(
    rows=1,
    cols=2,
    subplot_titles=[
        'Total Sales of Each Segment',
        'Average Sales of Each Segment'
    ]
)


fig.add_trace(
    go.Bar(
        x=segments,
        y=total_sales,
        marker_color='rgb(171,241,255)',
        marker_line_color='rgb(12,0,235)',
        marker_line_width=2,
        opacity=0.6
    ),
    row=1, col=1
)

fig.add_trace(
    go.Bar(
        x=segments,
        y=average_sales,
        marker_color='rgb(171,241,255)',
        marker_line_color='rgb(12,0,235)',
        marker_line_width=2,
        opacity=0.6
    ),
    row=1, col=2
)


fig.update_layout(
    template='simple_white',
    title='Sales Analysis by Segment',
    title_font_size=20,
    showlegend=False
)

fig.show()
```




```
total_profit = [df[df['Segment'] == i]['Profit'].sum() for i in segments]
average_profit = [df[df['Segment'] == i]['Profit'].mean() for i in segments]


fig = make_subplots(
    rows=1,
    cols=2,
    subplot_titles=[
        'Total Profit of Each Segment',
        'Average Profit of Each Segment'
    ]
)

fig.add_trace(
    go.Bar(
        x=segments,
        y=total_profit,
        marker_color='rgb(255,183,183)',
        marker_line_color='rgb(200,0,0)',
        marker_line_width=2,
        opacity=0.6
    ),
    row=1, col=1
)

fig.add_trace(
    go.Bar(
        x=segments,
        y=average_profit,
        marker_color='rgb(255,183,183)',
        marker_line_color='rgb(200,0,0)',
        marker_line_width=2,
        opacity=0.6
    ),
    row=1, col=2
)

fig.update_layout(
    template='simple_white',
    title='Profit Analysis by Segment',
    title_font_size=20,
    showlegend=False
)

fig.show()
```




```
df_scatter = df[['Sales', 'Profit', 'Segment']].copy()


plt.figure(figsize=(30, 8))

sns.scatterplot(
    x=df_scatter['Sales'],
    y=df_scatter['Profit'],
    hue=df_scatter['Segment'],
    palette=['#4A90E2', '#F5A623', '#7ED321'],
    size=df_scatter['Sales'],
    sizes=(100, 1000),
    legend='auto'
)

plt.title("Sales vs Profit Across Different Customer Segments", size=20, pad=20)

plt.show()
```



sllla

Analyze the Orders & Sales

```
df_grouped = df.groupby('Segment').agg({
    'Sales': 'sum',
    'Profit': ['sum', 'mean']
}).reset_index()
```

```
df_grouped.columns = ['Segment', 'Total Sales', 'Total Profit', 'Avg Profit']


fig = make_subplots(
    rows=2, cols=2,
    specs=[[{"type": "bar"}, {"type": "bar"}],
           [{"type": "scatter"}, {"type": "domain"}]],
    subplot_titles=[
        "Total Sales by Segment",
        "Total Profit by Segment",
        "Sales vs Profit",
        "Segment Contribution (Treemap)"
    ]
)


fig.add_trace(
    go.Bar(
        x=df_grouped['Segment'],
        y=df_grouped['Total Sales'],
        marker=dict(color='skyblue'),
        text=df_grouped['Total Sales'],
        textposition='outside'
    ),
    row=1, col=1
)

fig.add_trace(
    go.Bar(
        x=df_grouped['Segment'],
        y=df_grouped['Total Profit'],
        marker=dict(color='salmon'),
        text=df_grouped['Total Profit'],
        textposition='outside'
    ),
    row=1, col=2
)


fig.add_trace(
    go.Scatter(
        x=df['Sales'],
        y=df['Profit'],
        mode='markers',
        marker=dict(
            size=df['Sales']/50,
            color=df['Profit'],
            colorscale='Viridis',
            showscale=True
        ),
        text=df['Segment']
    ),
    row=2, col=1
)


treemap = px.treemap(
    df,
    path=['Segment'],
    values='Sales'
)

fig.add_trace(
```